

Log Monitoring & Alert System
Cloud Computing Lab

Project Report

April 21, 2026

Contents

1	Abstract	2
2	Problem Statement	2
3	Objectives	2
4	System Architecture	2
5	Core Components	2
5.1	Backend API Server	2
5.2	Anomaly Detection	3
5.3	Alert System	3
5.4	Log Generator	3
5.5	Dashboard	3
5.6	Serverless Processor	3
6	API Summary	3
7	Data Flow	4
8	Screenshots	4
9	Deployment and Infrastructure	4
9.1	Local Development	4
9.2	Docker	5
9.3	Terraform (AWS)	5
9.4	Kubernetes	5
10	Testing	5
11	Performance	6
12	Technology Stack	6
13	Limitations	6
14	Future Enhancements	7
15	Conclusion	7

1 Abstract

This project delivers a real-time log monitoring and alerting platform that ingests logs from multiple sources, detects anomalies using statistical methods, and visualizes system activity through a live dashboard. The system is designed for high throughput, low latency, and cloud-native deployment with automated scaling and alert escalation.

2 Problem Statement

Modern distributed systems generate high-volume logs across services and environments. Teams face delayed incident detection, noisy alerting from static thresholds, and fragmented log storage. The goal is to build an integrated platform that unifies log ingestion, provides real-time visibility, detects anomalies automatically, and triggers actionable alerts.

3 Objectives

1. Collect logs from multiple services in real time.
2. Store and process logs efficiently with a hybrid of in-memory buffering and persistent storage.
3. Visualize logs and metrics via a live interactive dashboard.
4. Detect anomalies using Z-score and IQR based methods.
5. Generate alerts through AWS SNS and Slack channels with escalation rules.
6. Support reproducible deployment on cloud and container platforms.

4 System Architecture

The system follows a layered pipeline:

- Layer 1. **Log Sources:** HTTP services, Syslog UDP, and Lambda bulk ingestion.
- Layer 2. **Ingest & Processing:** In-memory ring buffer with log enrichment and WebSocket broadcast.
- Layer 3. **Anomaly Detection:** Z-score for error spikes, IQR for slow response outliers.
- Layer 4. **Storage:** JSON-lines files with rotation, CloudWatch, and S3.
- Layer 5. **Visualization:** REST APIs and WebSocket stream feeding the dashboard.
- Layer 6. **Alerting:** Rule-based engine with escalation and multi-channel delivery.

5 Core Components

5.1 Backend API Server

The backend is an Express-based Node.js server that:

- Ingests logs via HTTP, Syslog UDP, and bulk endpoints.
- Enriches logs with metadata and persists them to JSON-lines files.
- Maintains a 5,000-entry ring buffer for fast queries.

- Exposes REST APIs for metrics, statistics, traces, and anomalies.
- Broadcasts real-time logs and anomaly updates through WebSockets.

5.2 Anomaly Detection

Two statistical detectors are used:

- **Z-score:** Detects sudden error spikes based on a 20-minute rolling baseline.
- **IQR:** Flags response-time outliers using a 500-sample history.

5.3 Alert System

A dedicated alert engine evaluates nine rules including error rate, latency percentiles, error velocity, and anomaly flags. Alerts are throttled by cooldown windows, escalated from WARNING to CRITICAL after repeated violations, and sent through AWS SNS and Slack webhooks.

5.4 Log Generator

A traffic simulator generates logs for five microservices with weighted severity levels. It injects cascade failures and anomaly bursts to exercise detection logic and alert thresholds.

5.5 Dashboard

The dashboard is a static HTML page that displays:

- Live log stream with pause and clear controls.
- Overview metrics including totals, error rates, and latency percentiles.
- Anomaly feed, heatmap visualization, and trace lookup.

5.6 Serverless Processor

The Lambda function processes CloudWatch log events, enriches them with a severity score, writes JSON-lines records to S3, and triggers SNS alerts when thresholds are exceeded.

6 API Summary

Key endpoints include:

- GET /health and GET /api/health/deep
- GET /api/metrics, /api/stats, /api/logs, /api/anomalies
- GET /api/heatmap, /api/top-errors, /api/trace/:id
- POST /api/ingest/bulk
- WS /ws/logs

7 Data Flow

1. Logs arrive from HTTP, Syslog, or Lambda sources.
2. Logs are enriched and appended to JSON-lines files.
3. Entries are pushed into the ring buffer.
4. Anomaly checks run and store results.
5. WebSocket broadcasts logs and anomalies to connected clients.
6. Alert engine polls metrics and fires alerts as needed.

8 Screenshots

Screenshots captured from the HTML mockups `Screenshot1.html`, `Screenshot2.html`, and `Screenshot3.html`.



Figure 1: Overview dashboard with live metrics and throughput trends.

9 Deployment and Infrastructure

9.1 Local Development

```
cd backend
npm install
npm start
npm run generate
npm run alerts
```

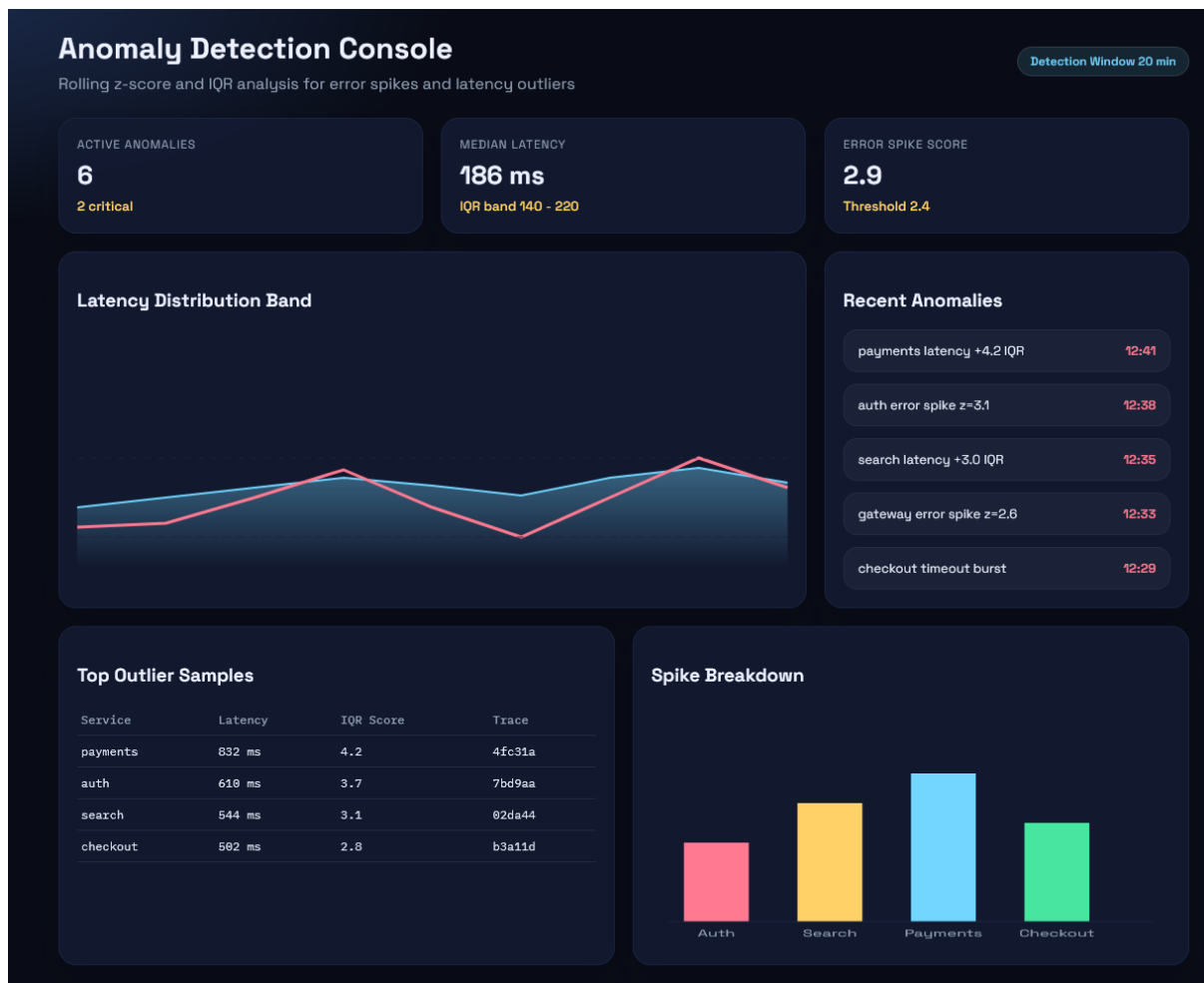


Figure 2: Anomaly detection view with spike analysis and timeline.

9.2 Docker

Docker Compose provisions the backend, generator, and alert services with shared log storage.

9.3 Terraform (AWS)

Infrastructure as code provisions a VPC, subnets, ALB, auto-scaling EC2 instances, S3 log storage, SNS topics, and CloudWatch alarms.

9.4 Kubernetes

The system can be deployed on Kubernetes using a Deployment with rolling updates, HPA scaling, ConfigMaps, Secrets, and an Ingress configured for WebSocket traffic.

10 Testing

- Integration tests validate ingestion, detection, and alerting logic.
- UI scripts test dashboard controls and API responses.
- Manual health checks verify readiness and anomaly endpoints.

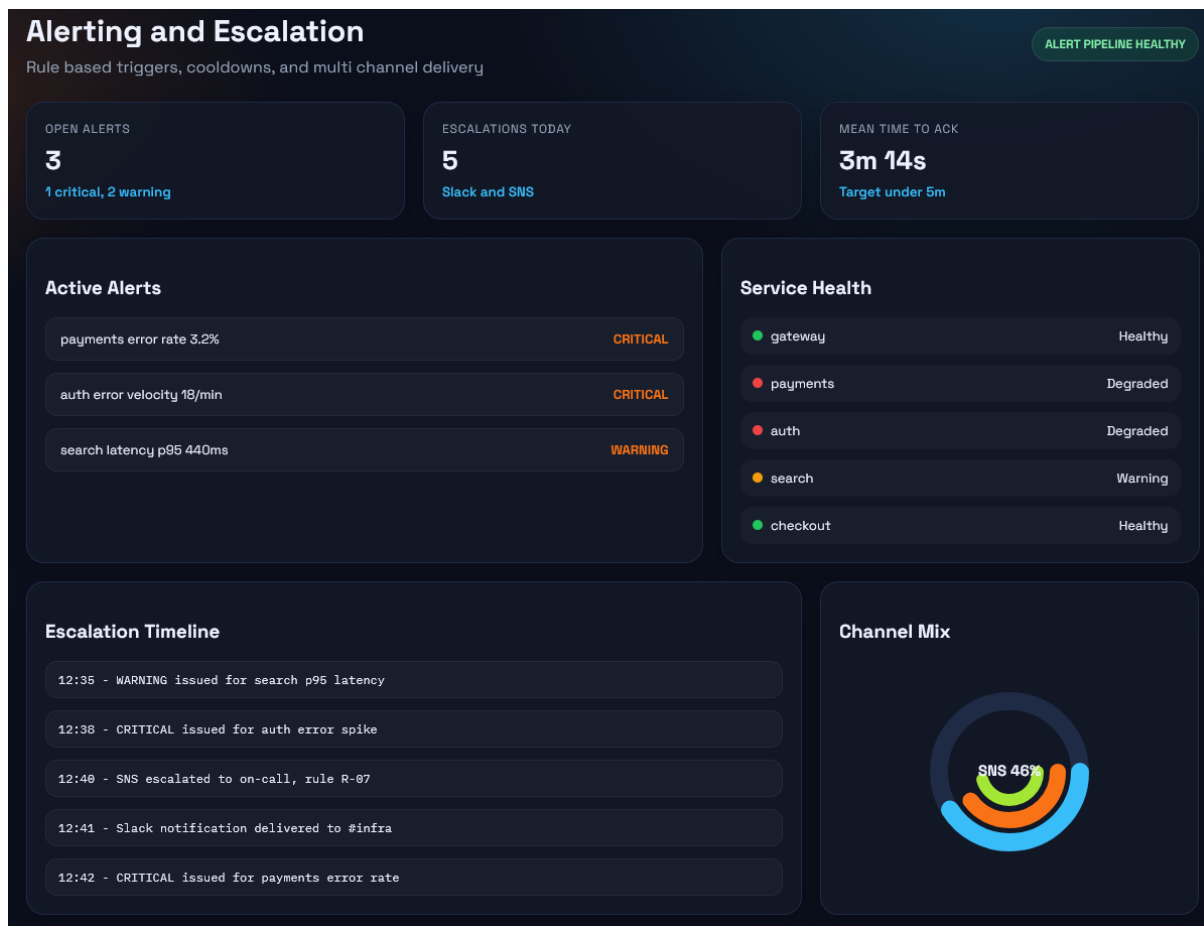


Figure 3: Alerting and service health status with escalation history.

11 Performance

Measured characteristics include sub-5 ms log write latency, sub-50 ms WebSocket broadcast, a 5,000-entry buffer capacity, and a sustained ingestion rate of roughly 500 logs per second on a single instance.

12 Technology Stack

Category	Technology
Runtime	Node.js 20
Web Framework	Express
Real-time	WebSocket (ws)
Cloud	AWS EC2, Lambda, CloudWatch, S3, SNS
IaC	Terraform
Orchestration	Docker, Kubernetes
Testing	Jest, Playwright

13 Limitations

- Rule-based detection may miss complex anomalies without adaptive ML.

- Single-region deployment can be a bottleneck during large outages.
- The dashboard currently focuses on operational metrics rather than deep log analytics.

14 Future Enhancements

1. Machine learning anomaly detection (e.g., Isolation Forest, LSTM).
2. Elasticsearch and Kibana integration for advanced search.
3. OpenTelemetry-based distributed tracing.
4. Log sampling and retention policies for cost optimization.
5. Multi-region deployment with active-active redundancy.
6. Role-based access control for dashboard access.

15 Conclusion

The Log Monitoring & Alert System integrates ingestion, anomaly detection, visualization, and alerting into a single platform. It provides a practical foundation for real-time observability and can be deployed locally, in containers, or on AWS with auto-scaling. The system demonstrates a complete monitoring workflow and is extensible for production-grade enhancements.